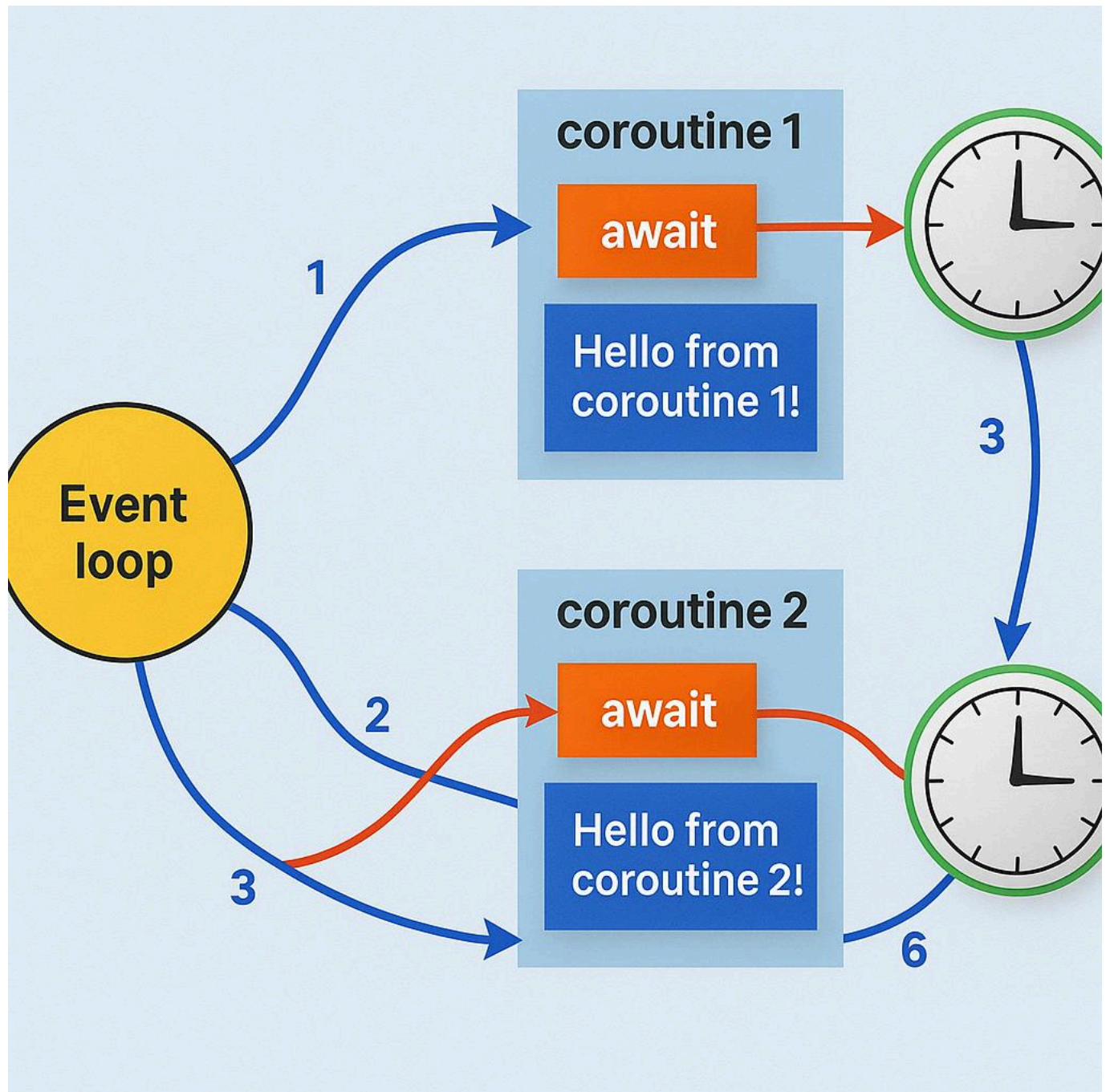


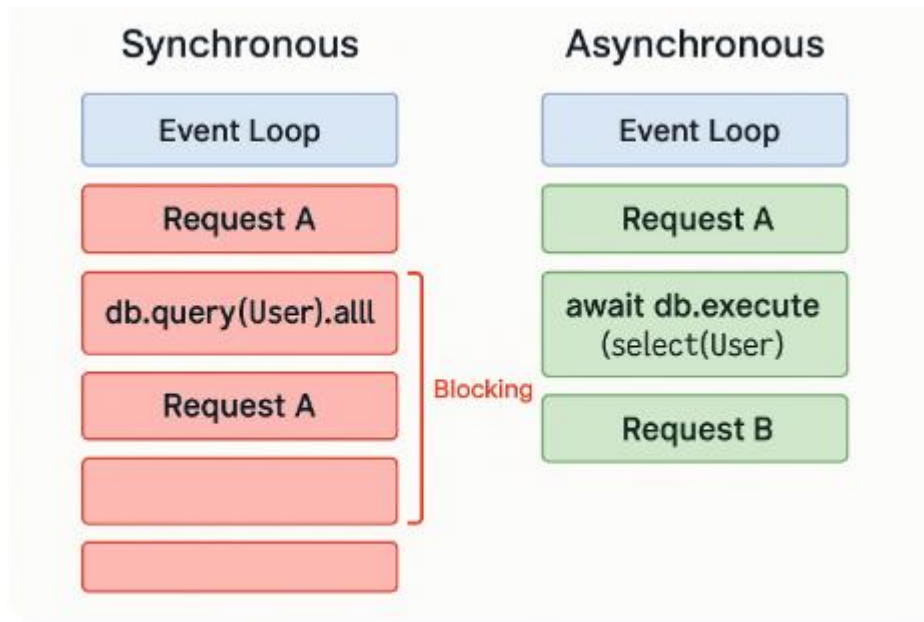
PART 7 — Async Python ⚡

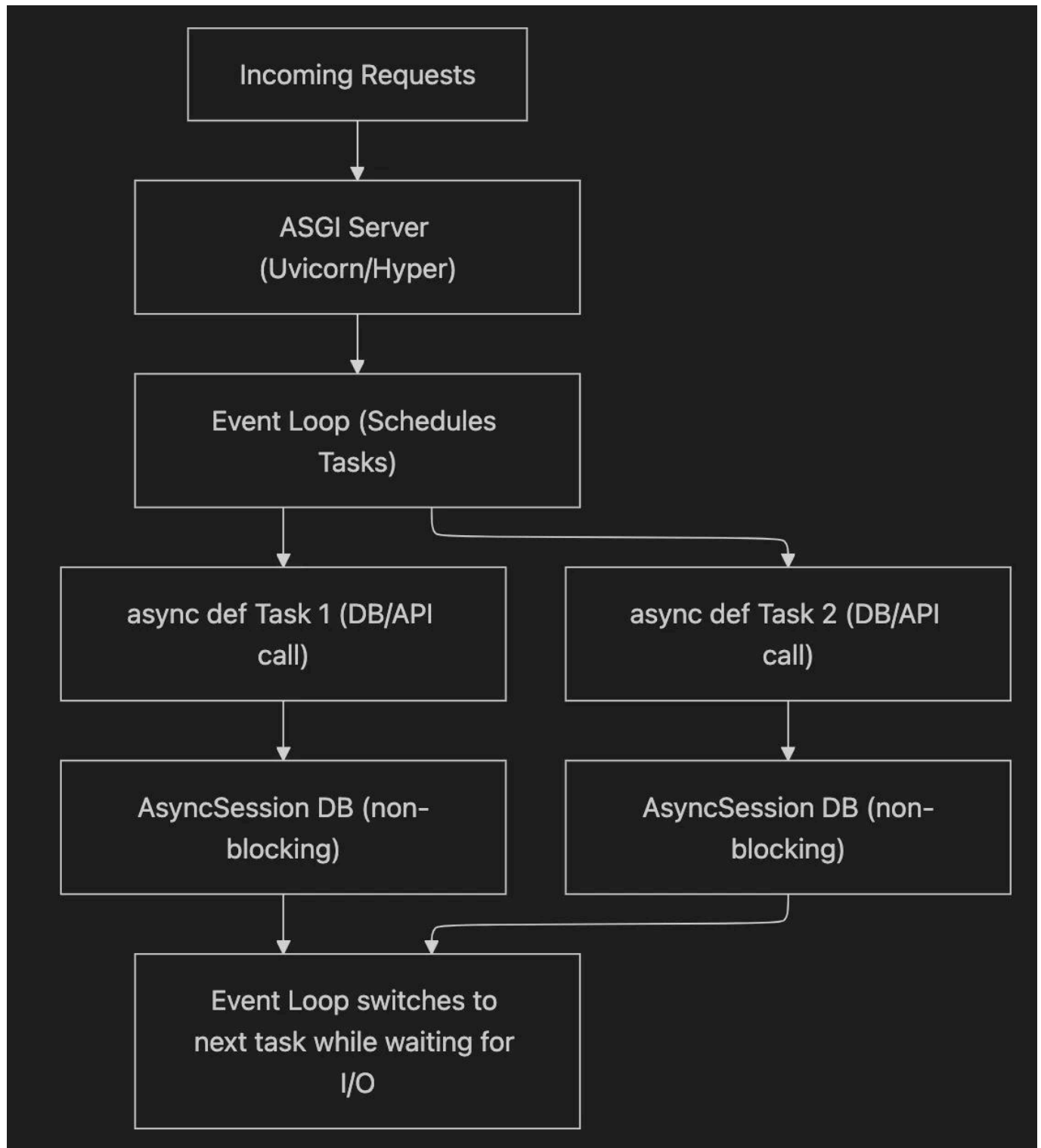
Especially important for FastAPI and high-concurrency backend systems

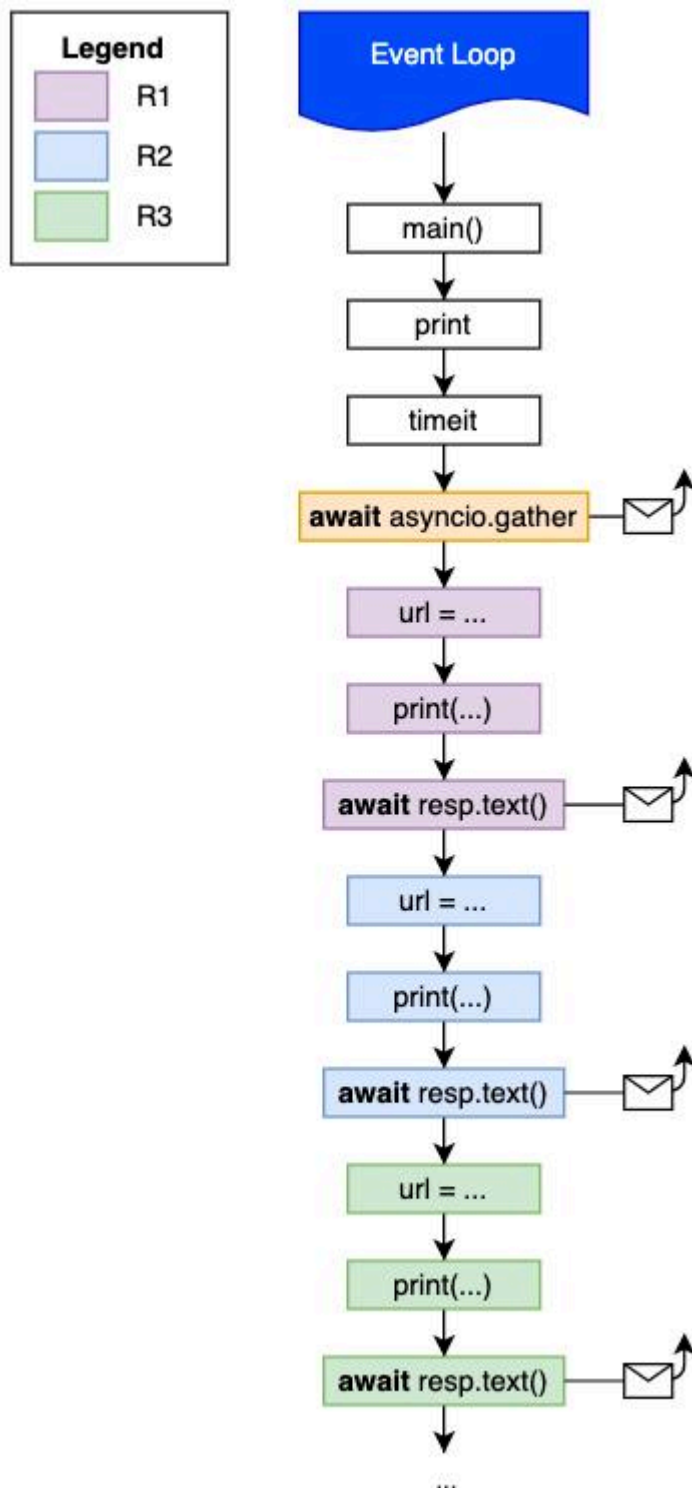
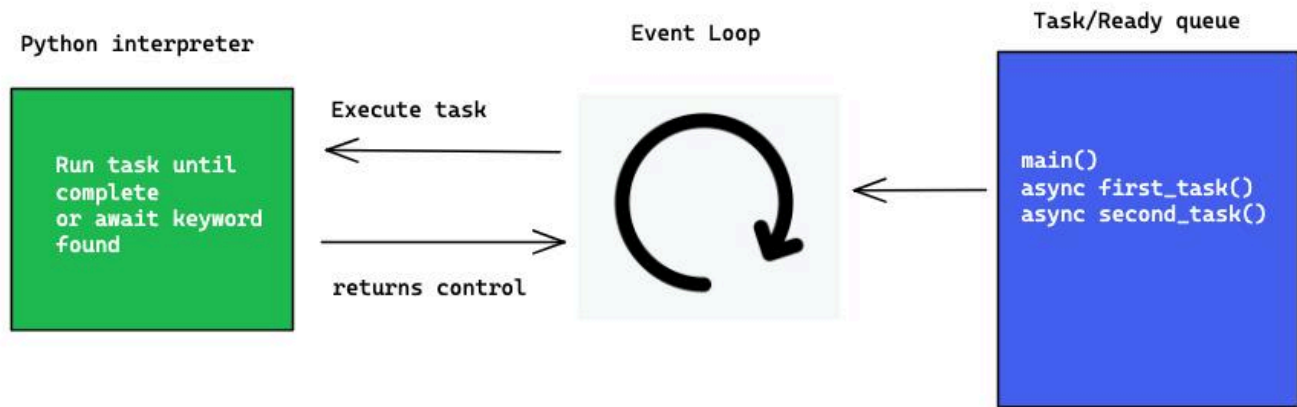
The **single most important mental model** is:

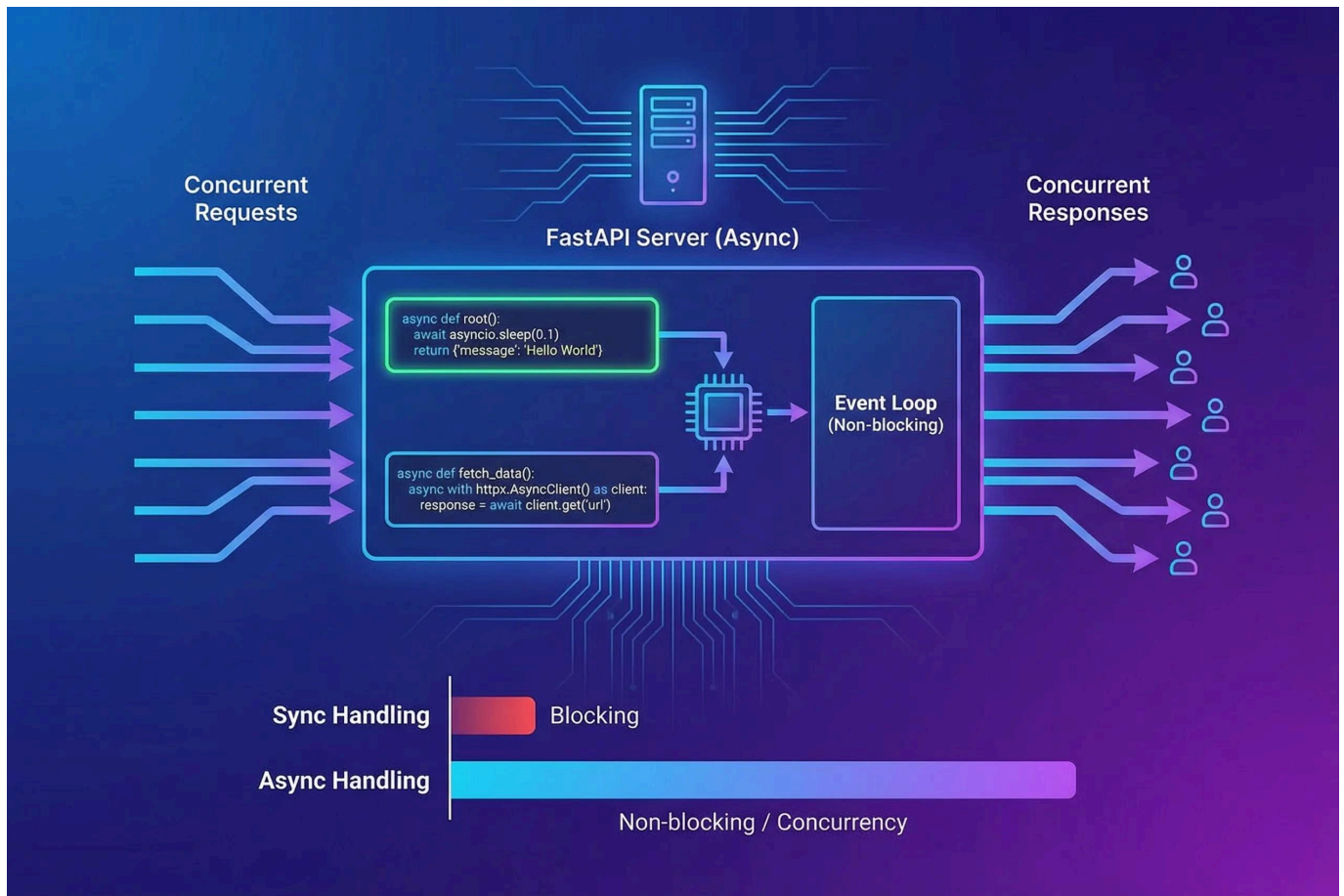
Async Python is about efficiently handling waiting, not making Python code magically faster.











MASTER MENTAL MODEL

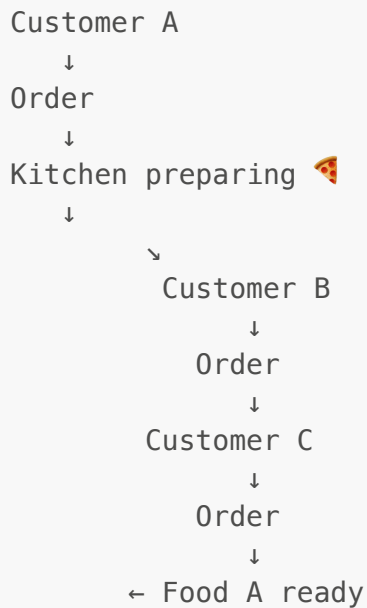
Think about a restaurant:

Synchronous

```
Customer A  
↓  
Order  
↓  
Wait for food 🍕  
↓  
Food arrives  
↓  
Customer B  
↓  
Order  
↓  
Wait...
```

The waiter is **stuck waiting**.

Asynchronous



The waiter doesn't sit idle while food is being prepared.

That's essentially what an **event loop** does.

1. Synchronous vs Asynchronous

Synchronous

One operation completes before the next starts.

```
def get_user():
    user = fetch_user()
    orders = fetch_orders()
    return user, orders
```

Conceptually:

```

fetch_user
██████████

fetch_orders
          ██████████

Total:
████████████████████
  
```

Asynchronous

While one operation is waiting, another can make progress.

```
async def get_data():
    user = await fetch_user()
    orders = await fetch_orders()
    return user, orders
```

If operations are independent:

```
async def get_data():
    user, orders = await asyncio.gather(
        fetch_user(),
        fetch_orders()
    )
```

Now:

```
fetch_user
██████████

fetch_orders
██████████

Total ≈ max(user, orders)
```

`asyncio` is specifically designed for concurrent asynchronous I/O and is commonly used underneath high-performance network and web frameworks. ([Python documentation](#))

2. Blocking vs Non-blocking

This distinction is **extremely important**.

Blocking

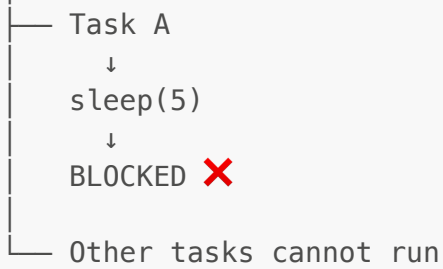
The current execution cannot make progress until the operation finishes.

```
import time

time.sleep(5)
```

Conceptually:

Event Loop



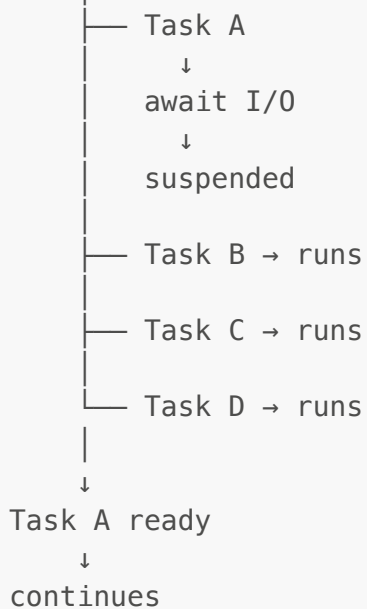
Non-blocking

The operation says:

"I'm waiting. You can work on something else."

```
await asyncio.sleep(5)
```

Event Loop



This is the heart of async programming.



FastAPI Connection

Consider:


```
@app.get("/users")
async def users():
    data = await database.fetch(...)
    return data
```

While the database is responding:

```
Request A
  ↓
await database
  ↓
waiting...

      Event Loop
        ↓
Request B —————→ executes
Request C —————→ executes
Request D —————→ executes

Database response
  ↓
Request A resumes
```

That's why async endpoints are powerful for **I/O-bound workloads**.

3. Coroutine

A coroutine is code that can **pause and resume**.

```
async def hello():
    await asyncio.sleep(1)
    return "Hello"
```

Calling it:

```
result = hello()
```

does **not** immediately execute the body.

You get a coroutine object.

```
async def hello()
    ↓
hello()
    ↓
Coroutine Object
    ↓
await / schedule
    ↓
actually runs
```

Python's docs distinguish between a **coroutine function** (`async def`) and the **coroutine object** produced when you call it. ([Python documentation](#))

4. `async`

`async` defines asynchronous code.

```
async def fetch_user():
    ...
```

It means:

"This function is a coroutine function and can use `await`."

Compare:

```
def normal():
    return 10
```

vs

```
async def async_func():
    return 10
```

Calling:

```
normal()
```

returns:

```
10
```

Calling:

```
async_func()
```

returns:

```
Coroutine object
```

5. `await`

`await` means:

Pause this coroutine until this awaitable is ready, while allowing the event loop to run other work.

Example:

```
async def get_user():  
    user = await database.fetch_user()  
    return user
```

Mental model:

```
Task  
↓  
await database  
↓  
PAUSE  
↓  
Event Loop runs another task  
↓  
database ready  
↓  
RESUME  
↓  
return user
```

Important:

```
await
```

does **not** mean:

```
"Stop the entire Python application."
```

It means:

```
"Suspend this coroutine."
```

6. `asyncio`

`asyncio` is Python's standard asynchronous programming library.

```
import asyncio
```

It provides:

- event loops
- coroutines
- Tasks
- Futures
- synchronization
- networking
- subprocess support
- queues
- timeouts
- cancellation

([Python documentation](#))

Typical program:

```
import asyncio

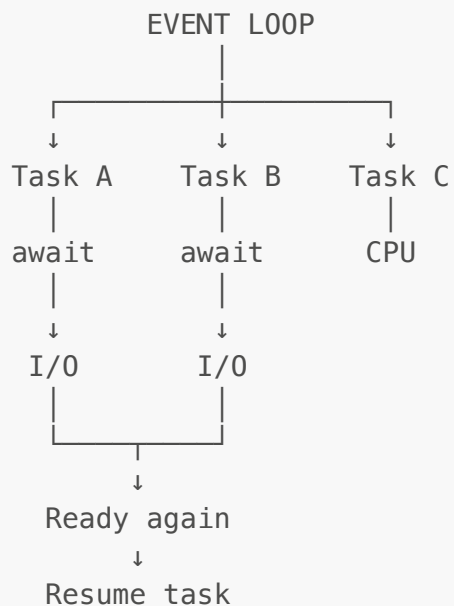
async def main():
    print("Hello")
    await asyncio.sleep(1)
    print("World")

asyncio.run(main())
```

7. Event Loop

This is the **heart of asyncio**.

Think:



The event loop repeatedly asks:

"Which task is ready to run?"

A simplified lifecycle:

```
Coroutine
  ↓
Task
  ↓
Event Loop
  ↓
Run coroutine
  ↓
await something
  ↓
Coroutine suspended
  ↓
Event Loop runs another task
  ↓
I/O completes
  ↓
Task becomes runnable
  ↓
Coroutine resumes
```

Asyncio uses **cooperative scheduling**: the event loop runs one task at a time, and when that task awaits an incomplete operation, another task can run. ([Python documentation](#))

8. Task

A **Task** is essentially a coroutine that has been scheduled to run by the event loop.

```
async def work():  
    await asyncio.sleep(2)  
    return "done"
```

Coroutine:

```
coro = work()
```

Task:

```
task = asyncio.create_task(work())
```

Mental model:

```
Coroutine  
  |  
  | create_task()  
  v  
Task  
  |  
  v  
Event Loop  
  |  
  v  
execution
```

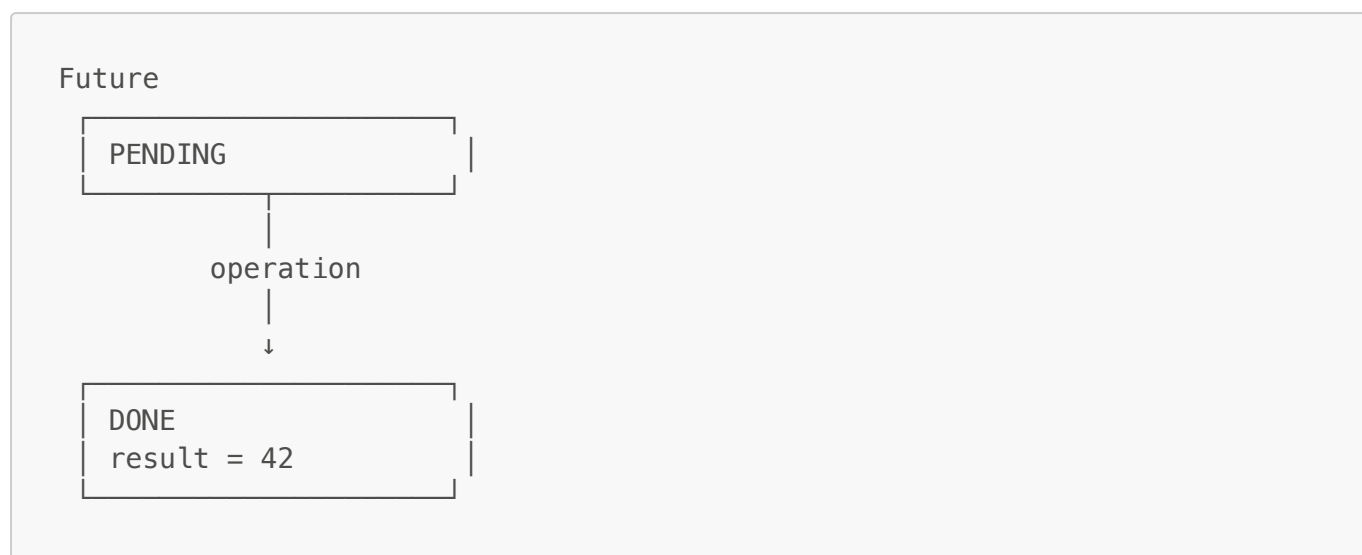
A Task lets the coroutine run concurrently with other Tasks. ([Python documentation](#))

9. Future

A **Future** represents:

A result that will be available later.

Think:



Example conceptually:

```

future = loop.create_future()

# Later:
future.set_result(42)

result = await future
  
```

Coroutine vs Task vs Future

Concept	Meaning
Coroutine	Async code that can pause/resume
Task	Scheduled coroutine
Future	Placeholder for eventual result

All three are **awaitable** in asyncio. ([Python documentation](#))

Senior interview point

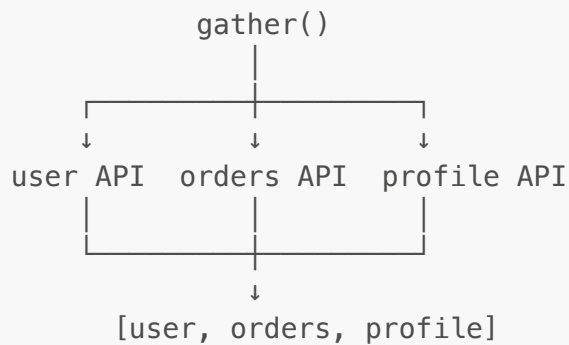
You normally **don't create Futures yourself** in application code. They are primarily a lower-level mechanism used to connect callback-based or event-loop APIs with async/await. ([Python documentation](#))

10. `gather()`

`asyncio.gather()` runs multiple awaitables concurrently and collects their results.

```
results = await asyncio.gather(
    fetch_user(),
    fetch_orders(),
    fetch_profile()
)
```

Think:



If:

```
user      = 2 sec
orders    = 3 sec
profile   = 1 sec
```

Sequential:

```
2 + 3 + 1 = 6 sec
```

Concurrent:

```
max(2, 3, 1) ≈ 3 sec
```

`gather()` preserves the order of the supplied awaitables in its result list. By default, an exception is propagated, while other awaitables aren't automatically cancelled merely because one raises. ([Python documentation](#))

11. `create_task()`

`create_task()` schedules a coroutine immediately for concurrent execution.


```
task1 = asyncio.create_task(fetch_user())
task2 = asyncio.create_task(fetch_orders())

user = await task1
orders = await task2
```

Compare:

Sequential

```
user = await fetch_user()
orders = await fetch_orders()
```

Concurrent

```
task1 = asyncio.create_task(fetch_user())
task2 = asyncio.create_task(fetch_orders())

user = await task1
orders = await task2
```

Visual:

```
create_task(fetch_user())
  ↓
Task A  —————→

create_task(fetch_orders())
  ↓
Task B  —————→

EVENT LOOP
  ↓
A and B progress
```

Python's current docs also recommend keeping a strong reference to tasks; for structured concurrency, `TaskGroup` is generally the more robust modern alternative. ([Python documentation](#))

gather() vs create_task()

This is a **very common interview question**.

create_task()

You want to **schedule a coroutine as a Task**.

```
task = asyncio.create_task(work())
```

gather()

You want to **run multiple awaitables concurrently and collect their results**.

```
results = await asyncio.gather(
    work1(),
    work2(),
    work3()
)
```

Think:

```
create_task()
  ↓
Schedule one coroutine
  ↓
Task

gather()
  ↓
Multiple awaitables
  ↓
Concurrent execution
  ↓
Results list
```

★ Modern Python: TaskGroup

For senior-level Python, know this too.

```
async with asyncio.TaskGroup() as tg:
    task1 = tg.create_task(fetch_user())
    task2 = tg.create_task(fetch_orders())
```

When the context exits, the tasks are awaited.

`TaskGroup` provides stronger structured-concurrency behavior than `gather()`, particularly around failures and cancellation. ([Python documentation](#))

12. Async Context Manager

Normal context manager:

```
with resource():  
    ...
```

Async context manager:

```
async with resource():  
    ...
```

It uses:

```
__aenter__()  
__aexit__()
```

Example:

```
class Connection:  
    async def __aenter__(self):  
        await self.connect()  
        return self  
  
    async def __aexit__(self, exc_type, exc, tb):  
        await self.close()
```

Usage:

```
async with Connection() as conn:  
    await conn.query()
```

Mental model:

```
async with  
↓
```

```
__aenter__()  
  ↓  
resource acquired  
  ↓  
use resource  
  ↓  
__aexit__()  
  ↓  
resource released
```

This is particularly useful for:

- async database connections
- HTTP clients
- locks
- transactions
- streams

13. Async Generator

A normal generator:

```
def numbers():  
    yield 1  
    yield 2  
    yield 3
```

Async generator:

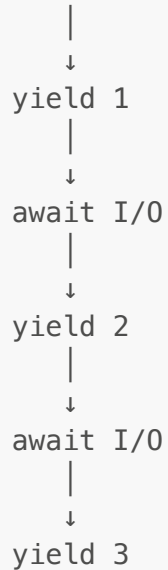
```
async def numbers():  
    yield 1  
    await asyncio.sleep(1)  
    yield 2  
    await asyncio.sleep(1)  
    yield 3
```

Consume it with:

```
async for number in numbers():  
    print(number)
```

Mental model:

Async Generator



```
graph TD; A[Async Generator] --> B[yield 1]; B --> C[await I/O]; C --> D[yield 2]; D --> E[await I/O]; E --> F[yield 3];
```

Very useful for:

- streaming responses
- database cursors
- paginated APIs
- large datasets
- WebSockets
- Server-Sent Events

14. Async Iterator

An object is an async iterator if it supports asynchronous iteration.

Conceptually:

```
class AsyncCounter:
    def __aiter__(self):
        return self

    async def __anext__(self):
        ...
```

Then:

```
async for value in AsyncCounter():
    print(value)
```

The protocol is:

```
async for
  ↓
__aiter__()
  ↓
__anext__()
  ↓
await result
  ↓
value
  ↓
repeat
  ↓
StopAsyncIteration
```

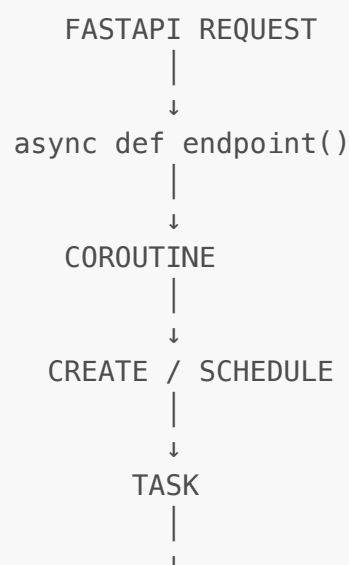
Compare:

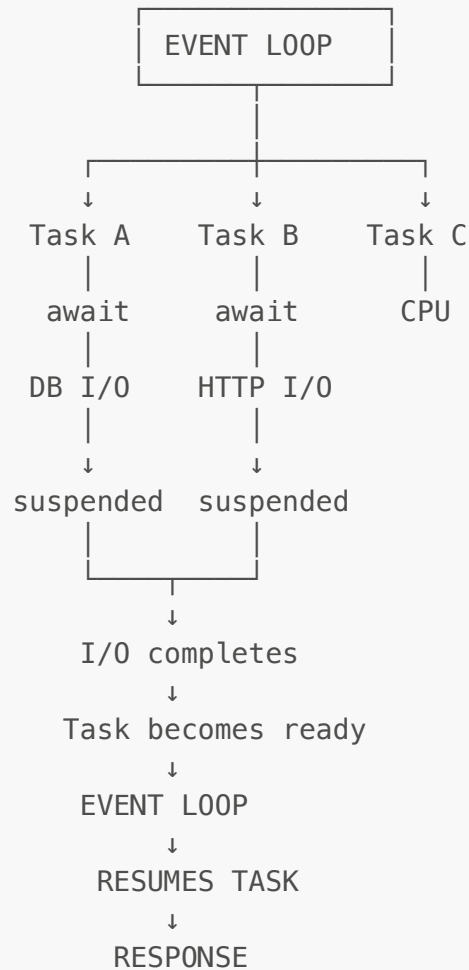
Normal	Async
<code>for</code>	<code>async for</code>
<code>__iter__()</code>	<code>__aiter__()</code>
<code>__next__()</code>	<code>__anext__()</code>
<code>StopIteration</code>	<code>StopAsyncIteration</code>
synchronous	can await



The Complete Async Architecture

This is the diagram I want you to remember for FastAPI:





The Biggest Async Mistake

This:

```

async def endpoint():
    time.sleep(5)
    return {"ok": True}
  
```

is **bad**.

Why?

Because `time.sleep()` blocks the thread/event loop.

Better:

```

async def endpoint():
    await asyncio.sleep(5)
    return {"ok": True}
  
```

The second version allows other tasks to run while waiting.



Another Critical Mistake

Don't assume:

```
async def
```

automatically makes everything asynchronous.

This is still blocking:

```
async def endpoint():  
    result = some_blocking_function()  
    return result
```

If `some_blocking_function()` performs blocking I/O or heavy synchronous work, it can block the event-loop thread.

The important question isn't:

```
"Is this function declared async?"
```

It's:

```
"Does the operation yield control while waiting?"
```



CPU-Bound vs I/O-Bound

This is critical for FastAPI interviews.

I/O-bound

Examples:

```
Database  
HTTP API  
Redis  
File/network I/O  
Waiting for external service
```

Async is often excellent here:


```
Task A → await DB
Task B → await HTTP
Task C → await Redis
```

CPU-bound

Examples:

```
Image processing
Huge calculations
ML computation
Compression
Video processing
```

Async doesn't magically make CPU work parallel.

```
Task A
████████████████████
      ↓
Event loop can't do useful work
```

For CPU-heavy workloads, consider:

- multiprocessing
- process pools
- native extensions
- external workers
- specialized compute services

FastAPI Mental Model

When you write:

```
@app.get("/users")
async def get_users():
    users = await db.fetch_users()
    return users
```

Think:

```
HTTP Request
  ↓
FastAPI
  ↓
async endpoint
  ↓
Coroutine
  ↓
Task
  ↓
Event Loop
  ↓
await DB
  ↓
Task pauses
  ↓
Other requests execute
  ↓
DB response arrives
  ↓
Task resumes
  ↓
HTTP Response
```

That's the architecture you should be able to explain in a **senior FastAPI interview**.

One-Line Definitions — Interview Cheat Sheet

Topic	Remember
Synchronous	One operation waits for the previous one
Asynchronous	Waiting work can yield so other work progresses
Blocking	Prevents current execution from progressing
Non-blocking	Allows other work while waiting
Coroutine	Async computation that can suspend/resume
<code>async</code>	Defines coroutine function
<code>await</code>	Suspends current coroutine until awaitable progresses/completes
<code>asyncio</code>	Python's async/concurrency framework
Event Loop	Schedules and runs async tasks
Task	Scheduled coroutine
Future	Placeholder for eventual result

Topic	Remember
<code>gather()</code>	Run multiple awaitables concurrently + collect results
<code>create_task()</code>	Schedule coroutine as a Task
Async Context Manager	<code>async with</code> , using <code>__aenter__</code> / <code>__aexit__</code>
Async Generator	Async function using <code>yield</code>
Async Iterator	Implements <code>__aiter__</code> / <code>__anext__</code>

Senior Interview Questions You Should Be Able to Answer

Basic

1. What is the difference between synchronous and asynchronous programming?
2. What is a coroutine?
3. What does `await` actually do?
4. What is an event loop?
5. What is a Task?
6. What is a Future?

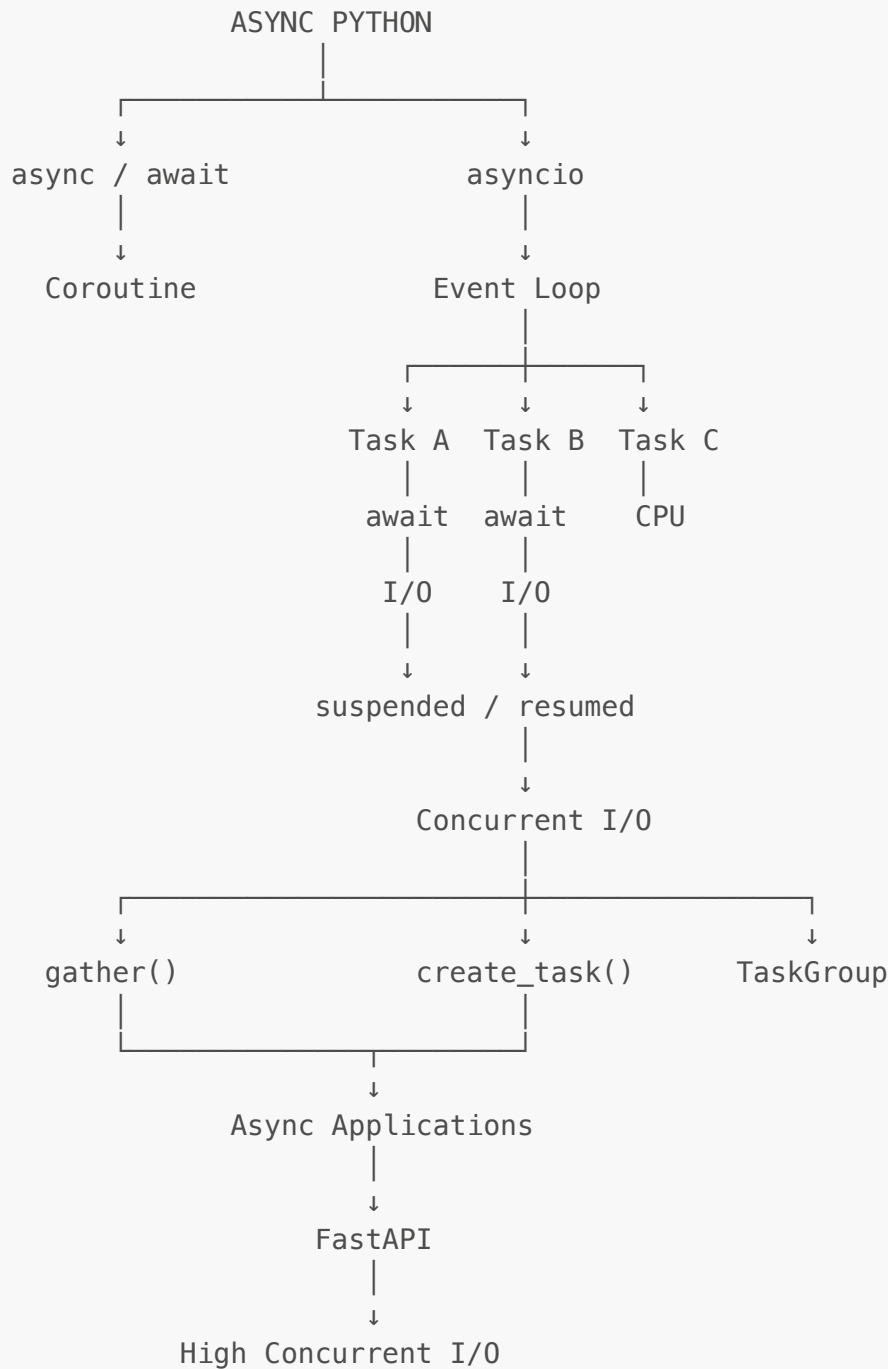
Intermediate

7. Difference between coroutine and Task?
8. Difference between `gather()` and `create_task()`?
9. Why does `time.sleep()` block an async application?
10. Why is `asyncio.sleep()` different?
11. When should you use async?
12. When should you **not** use async?
13. What happens when an exception occurs inside `gather()`?
14. What is an async context manager?
15. What is an async generator?

Senior / FastAPI

16. How does FastAPI execute `async def` endpoints?
17. What happens to the event loop when your endpoint performs blocking I/O?
18. Why can one blocking operation affect many concurrent requests?
19. How do you handle a synchronous library inside an async endpoint?
20. Difference between I/O-bound and CPU-bound workloads?
21. `gather()` vs `TaskGroup`?
22. How does cancellation propagate through async Tasks?
23. How would you prevent a slow external API from blocking your application?
24. How would you stream a large response asynchronously?

Final Mental Map



The 5 things to remember most

1. **async def** → coroutine
2. **await** → give control back while waiting
3. Event loop → schedules/resumes tasks
4. Task → scheduled coroutine
5. Async shines primarily for I/O-bound concurrency

The current Python documentation describes coroutines, Tasks, and Futures as the three major awaitable types, and notes that the event loop uses cooperative scheduling. ([Python documentation](#))

One senior-level correction to keep in your head: async does **not** mean “parallel.” It is primarily **concurrency through cooperative scheduling**; parallel CPU execution is a separate concern.